

06_results_summary

March 27, 2026

1 06 — Oppsummering av resultater

Sammenstiller alle resultater fra scenariosimuleringen: koeffisientestimater, ΔP per modell og sesong, sårbarhetsmål og konvergens.

Seksjon A — Sesongkoeffisienter ($\hat{Q}$) **Seksjon B** — Priseffekt: ΔP per modell og sesong

Seksjon C — Konvergenstabell **Seksjon D** — Sårbarhetsmål (ISO-regimeandel, ekstrempriser)

Seksjon E — Figur: ΔP per sesong (alle modeller) **Seksjon F** — Husholdningskostnadseffekt

Input: intermediate/scenarios.pkl, intermediate/df_iso.parquet,
intermediate/models.pkl, intermediate/models_ols_seasonal.pkl,
intermediate/convergence_table.csv

```
[17]: import os, warnings, pickle
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm

from src.config import (
    INTERMEDIATE_DIR, FIG_DIR, STARGATE_MW, STARGATE_TOTAL_MW,
    SEASON_ORDER, TARGET, DESEASON_FEATURES, DESEASON_TARGET,
    apply_style, C1, C2, C3, C4, C5, PALETTE
)

apply_style()
SEC_DIR = FIG_DIR + "section_06/"
os.makedirs(SEC_DIR, exist_ok=True)
```

```
[18]: # Scenariosimuleringer
with open(INTERMEDIATE_DIR + "scenarios.pkl", "rb") as f:
    scen_data = pickle.load(f)
scenarios      = scen_data["scenarios"]
MODEL_DP      = scen_data["MODEL_DP"]
iso_share_results = scen_data["iso_share_results"]
p90_threshold  = scen_data["p90_threshold"]
cv_beta_min    = scen_data["cv_beta_min"]
```

```

cv_beta_max      = scen_data["cv_beta_max"]

# OLS-modeller
with open(INTERMEDIATE_DIR + "models.pkl", "rb") as f:
    models = pickle.load(f)
ols_model = models["ols_model"]

# Sesongspesifikke OLS-modeller
with open(INTERMEDIATE_DIR + "models_ols_seasonal.pkl", "rb") as f:
    seasonal_data = pickle.load(f)
seasonal_models = seasonal_data["models"]

# ISO data
df_iso = pd.read_parquet(INTERMEDIATE_DIR + "df_iso.parquet")

SEASON_LABELS = {"Winter": "Vinter", "Spring": "Vår",
                  "Summer": "Sommer", "Autumn": "Høst"}
SEASON_COLS   = [C1, C2, C3, C4]

print(f"Modeller i MODEL_DP: {list(MODEL_DP.keys())}")
print(f"Scenariodeltaer:      {list(scenarios.keys())} MW")
print(f"df_iso:               {len(df_iso):,} ISO-timer")

```

```

Modeller i MODEL_DP: ['OLS (HAC)', 'OLS (sesong)', 'XGBoost', 'MS-DR']
Scenariodeltaer:      [230, 520] MW
df_iso:               34,916 ISO-timer

```

1.1 Seksjon A — Sesongkoeffisienter $\hat{_Q}$

Forbrukskoeffisienten ($\hat{_Q}$) angir prissensitivitet per MW i ISO-regime. OLS (HAC) estimerer én felles koeffisient; OLS (sesong) gir sesongspesifikke estimater.

```

[19]: #  $\hat{\_Q}$ : OLS (HAC) total + sesongspesifikke estimater
rows = []

# Felles OLS (HAC)
b_ols = ols_model.params["cons_avvik"]
se_ols = ols_model.bse["cons_avvik"]
rows.append({
    "Modell": "OLS (HAC) - total",
    "r_Q": round(b_ols, 4),
    "SE (HAC)": round(se_ols, 4),
    "95% KI": f"[{b_ols - 1.96*se_ols:.4f}, {b_ols + 1.96*se_ols:.4f}]",
    "ΔP +230 MW": round(b_ols * STARGATE_MW, 1),
})

```

```
# Sesongspesifikke
for season, label in SEASON_LABELS.items():
    res = seasonal_models[season]
    b = res.params["cons_avvik"]
    se = res.bse["cons_avvik"]
    ci = res.conf_int(alpha=0.05).loc["cons_avvik"]
    n = int(res.nobs)
    rows.append({
        "Modell": f"OLS (sesong) - {label}",
        "r_Q": round(b, 4),
        "SE (HAC)": round(se, 4),
        "95% KI": f"[{ci[0]:.4f}, {ci[1]:.4f}]",
        "ΔP +230 MW": round(b * STARGATE_MW, 1),
    })

coef_df = pd.DataFrame(rows).set_index("Modell")
print("r_Q - forbrukskoeffisient per sesong")
display(coef_df)
```

$\hat{r}_Q$ - forbrukskoeffisient per sesong

	$\hat{r}_Q$	SE (HAC)	95% KI	ΔP +230 MW
Modell				
OLS (HAC) - total	0.3060	0.0348	[0.2377, 0.3743]	70.4
OLS (sesong) - Vinter	0.7773	0.0977	[0.5858, 0.9688]	178.8
OLS (sesong) - Vår	0.3509	0.0461	[0.2606, 0.4411]	80.7
OLS (sesong) - Sommer	-0.1411	0.0315	[-0.2028, -0.0793]	-32.4
OLS (sesong) - Høst	0.1820	0.0576	[0.0692, 0.2949]	41.9

1.2 Seksjon B — Preiseffekt: ΔP per modell og sesong

Gjennomsnittlig prisøkning (NOK/MWh) i ISO-timer under hvert scenario.

```
[20]: for delta, label in [(STARGATE_MW, f"Fase 1 (+{STARGATE_MW} MW)"),
                           (STARGATE_TOTAL_MW, f"Fase 1+2 (+{STARGATE_TOTAL_MW}
                           ↪MW)"]):
    res = scenarios[delta]
    rows = []
    for name, col in MODEL_DP.items():
        row = {"Modell": name, "Total": round(res[col].mean(), 1)}
        for s_en, s_no in SEASON_ORDER.items():
            row[s_no] = round(res.loc[res["season"] == s_en, col].mean(), 1)
        rows.append(row)
    print(f"\nΔP (NOK/MWh) - {label}")
    print("=" * 65)
    display(pd.DataFrame(rows).set_index("Modell").style.format("{:.1f}"))
```

ΔP (NOK/MWh) - Fase 1 (+230 MW)

=====

<pandas.io.formats.style.Styler at 0x1330f8790>

ΔP (NOK/MWh) - Fase 1+2 (+520 MW)

=====

<pandas.io.formats.style.Styler at 0x1332fc150>

1.3 Seksjon C — Konvergenstabell

Estimert priseffekt (+230 MW) på tvers av alle fire modeller. Konvergens mellom modeller styrker robustheten til ΔP -estimaten.

```
[21]: # Last konvergenstabellen fra notebook 05, eller rekonstruer fra scenarios
try:
    conv_df = pd.read_csv(INTERMEDIATE_DIR + "convergence_table.csv",
                          index_col="Modell")
    print("Konvergenstabell (lastet fra fil):")
    display(conv_df)
except FileNotFoundError:
    # Rekonstruer enkelt
    res230 = scenarios[STARGATE_MW]
    rows = []
    for name, col in MODEL_DP.items():
        rows.append({"Modell": name,
                    "ΔP (NOK/MWh)": round(res230[col].mean(), 1)})
    print("Konvergenstabell (rekonstruert):")
    display(pd.DataFrame(rows).set_index("Modell"))

# Modellspenn
res230 = scenarios[STARGATE_MW]
dp_means = {name: res230[col].mean() for name, col in MODEL_DP.items()}
print(f"\nModellspenn {STARGATE_MW} MW: "
      f"[{min(dp_means.values()):+.1f}, {max(dp_means.values()):+.1f}] NOK/MWh")
res520 = scenarios[STARGATE_TOTAL_MW]
dp520 = {name: res520[col].mean() for name, col in MODEL_DP.items()}
print(f"Modellspenn {STARGATE_TOTAL_MW} MW: "
      f"[{min(dp520.values()):+.1f}, {max(dp520.values()):+.1f}] NOK/MWh")
```

Konvergenstabell (lastet fra fil):

	ΔP (NOK/MWh)	KI / spenn	Usikkerhetstype
Modell			
OLS (HAC)	70.400000	[54.7, 86.1]	95 % HAC KI
OLS (sesong)	70.000000	Vinter: 179 - Sommer: -32	Sesongspenn

XGBoost	74.099998	Vinter: 126 - Sommer: 47	Sesongspenn
MS-DR (vektet)	56.600000	[52.5, 60.8]	Regime-vektet 95 % KI

Modellspenn +230 MW: [+56.6, +74.1] NOK/MWh
Modellspenn +520 MW: [+128.0, +159.1] NOK/MWh

1.4 Seksjon D — Sårbarhetsmål

ISO-regimeandel: andel timer der NO4 er koblet fra det nordiske nettet (prisen settes lokalt).
Ekstrempriser: andel timer over P90-terskelen.

```
[22]: # ISO-regimeandel
iso_df = pd.DataFrame(iso_share_results).set_index("Scenario")
print(f"ISO-regimeandel (%) - P90-terskel: {p90_threshold:.0f} NOK/MWh")
print("=" * 65)
display(iso_df.style.format("{:.1f}"))

iso_base = iso_df.loc["Baseline", "Total"]
iso_230 = iso_df.loc[f"+{STARGATE_MW} MW", "Total"]
iso_520 = iso_df.loc[f"+{STARGATE_TOTAL_MW} MW", "Total"]
print(f"\nΔ ISO-andel: +{STARGATE_MW} MW → {iso_230 - iso_base:+.1f} pp, "
      f"+{STARGATE_TOTAL_MW} MW → {iso_520 - iso_base:+.1f} pp")

# Ekstrempriser (rekonstruert fra scenarios)
print(f"\nEkstrempriser > P90 ({p90_threshold:.0f} NOK/MWh)")
print("=" * 65)

ext_rows = []
for delta, label in [(0, "Baseline"), (STARGATE_MW, f"+{STARGATE_MW} MW"),
                     (STARGATE_TOTAL_MW, f"+{STARGATE_TOTAL_MW} MW")]:
    row = {"Scenario": label}
    for name, col in MODEL_DP.items():
        if delta == 0:
            prices = df_iso[TARGET]
        else:
            prices = scenarios[delta][f"P_scen_{col}"]
        row[name] = round(100 * (prices > p90_threshold).sum() / len(prices), 1)
    ext_rows.append(row)

ext_df = pd.DataFrame(ext_rows).set_index("Scenario")
display(ext_df.style.format("{:.1f}"))
```

ISO-regimeandel (%) - P90-terskel: 544 NOK/MWh

=====

<pandas.io.formats.style.Styler at 0x1330fbe50>

Δ ISO-andel: +230 MW $\rightarrow$ +3.5 pp, +520 MW $\rightarrow$ +7.5 pp

Ekstrempriser > P90 (544 NOK/MWh)

=====

<pandas.io.formats.style.Styler at 0x173b59950>

1.5 Seksjon E — Figur: ΔP per sesong (alle modeller, +230 MW)

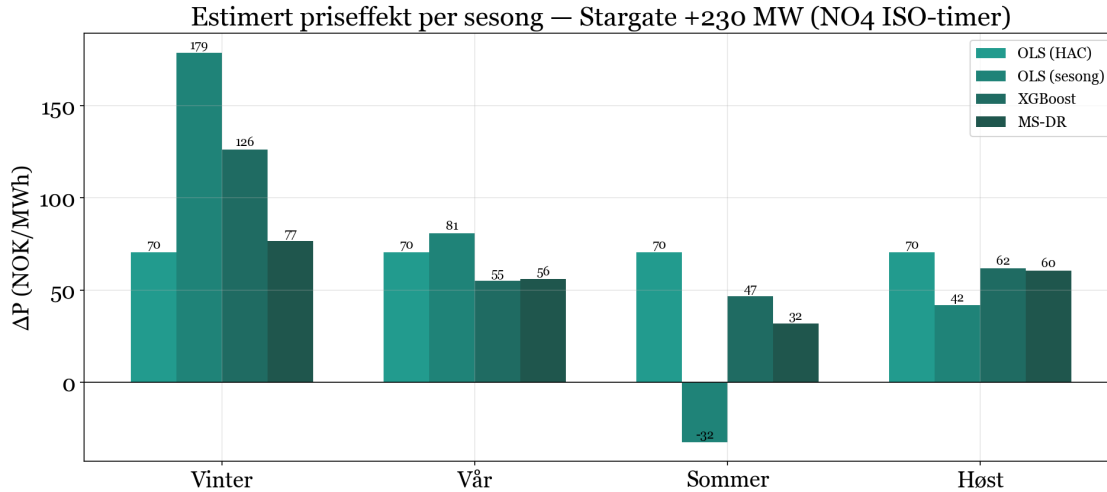
```
[23]: res = scenarios[STARGATE_MW]
seasons_no = [SEASON_LABELS[s] for s in SEASON_ORDER]
model_names = list(MODEL_DP.keys())
n_models = len(model_names)
x = np.arange(len(seasons_no))
width = 0.18

fig, ax = plt.subplots(figsize=(13, 6))

for i, (name, col) in enumerate(MODEL_DP.items()):
    vals = [res.loc[res["season"] == s_en, col].mean()
            for s_en in SEASON_ORDER]
    offset = (i - (n_models - 1) / 2) * width
    bars = ax.bar(x + offset, vals, width, label=name,
                  color=PALETTE[i % len(PALETTE)], alpha=0.88)
    for bar, v in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width() / 2,
                bar.get_height() + 0.3, f"{v:.0f}",
                ha="center", va="bottom", fontsize=10)

ax.set_xticks(x)
ax.set_xticklabels(seasons_no)
ax.set_ylabel(" $\Delta P$  (NOK/MWh)")
ax.set_title(f"Estimert priseffekt per sesong - Stargate +{STARGATE_MW} MW (NO4_
 $\hookrightarrow$ ISO-timer)")
ax.legend(fontsize=12, loc="upper right")
ax.axhline(0, color="black", lw=0.8)

plt.tight_layout()
plt.savefig(SEC_DIR + "dp_per_sesong_alle_modeller.pdf", bbox_inches="tight")
plt.show()
```



1.6 Seksjon F — Husholdningskostnadseffekt

Forenklet beregning: hva betyr ΔP for en gjennomsnittlig husholdning? Antar kun ISO-timer påvirkes (ISO-andel som vektfaktor).

```
[24]: HH_CONSUMPTION_MWH = 20          # 20 000 kWh/år
iso_fraction = iso_base / 100         # andel timer NO4 er isolert

rows = []
for delta, label in [(STARGATE_MW, f"+{STARGATE_MW} MW"),
                     (STARGATE_TOTAL_MW, f"+{STARGATE_TOTAL_MW} MW")]:
    dp_ols = scenarios[delta]["dp_OLS"].mean()
    cost = dp_ols * HH_CONSUMPTION_MWH * iso_fraction

    # OLS CV-intervall
    cost_lo = cv_beta_min * delta * HH_CONSUMPTION_MWH * iso_fraction
    cost_hi = cv_beta_max * delta * HH_CONSUMPTION_MWH * iso_fraction

    rows.append({
        "Scenario":          label,
        "ΔP (NOK/MWh)": f"{dp_ols:+.1f}",
        "ISO-andel (%)": f"{iso_base:.1f}",
        "Estimert kostnad (NOK/år)": f"{cost:+,.0f}",
        "CV-intervall (NOK/år)": f"[{cost_lo:+,.0f}, {cost_hi:+,.0f}]",
    })

print(f"Husholdningsforbruk: {HH_CONSUMPTION_MWH * 1000:,} kWh/år")
print(f"ISO-andel (baseline): {iso_base:.1f}% → vektfaktor = {iso_fraction:.3f}")
```

```
display(pd.DataFrame(rows).set_index("Scenario"))
print("\nMerk: forenklet beregning. Reell effekt avhenger av om husholdningen"
      " er på spotpriskontrakt og om NO4 er i ISO-regime.")
```

Husholdningsforbruk: 20,000 kWh/år

ISO-andel (baseline): 72.5% → vektfaktor = 0.725

	ΔP (NOK/MWh)	ISO-andel (%)	Estimert kostnad (NOK/år) \
Scenario			
+230 MW	+70.4	72.5	+1,020
+520 MW	+159.1	72.5	+2,307

	CV-intervall (NOK/år)
Scenario	
+230 MW	[+214, +1,020]
+520 MW	[+484, +2,307]

Merk: forenklet beregning. Reell effekt avhenger av om husholdningen er på spotpriskontrakt og om NO4 er i ISO-regime.